7. Multimodal Visual Understanding + SLAM Navigation + Visual Line Following

1. Course Content

1. Learn to use compound function cases such as the robot's visual understanding, visual line following, and SLAM navigation.
2. Learn the new key source code appearing in the tutorial.
3. **Note: This chapter requires completing the map mapping file configuration according to the previous chapter, Multimodal Visual Understanding + SLAM Navigation.**

2. Preparation

2.1 Content Description

> The virtual machine used by default with this product starts the Dify Docker environment automatically at boot, so no additional startup is usually needed.

⚠ The same test instructions will not produce exactly the same reply content from the large model, and will differ slightly from the screenshots in the tutorial. If you need to enhance or reduce the diversity of the large model's replies, refer to 【AI Large Model Development】-【RAG Knowledge Base Deployment】 to modify the knowledge base and sample library.

Open the virtual machine terminal, start the Dify environment:

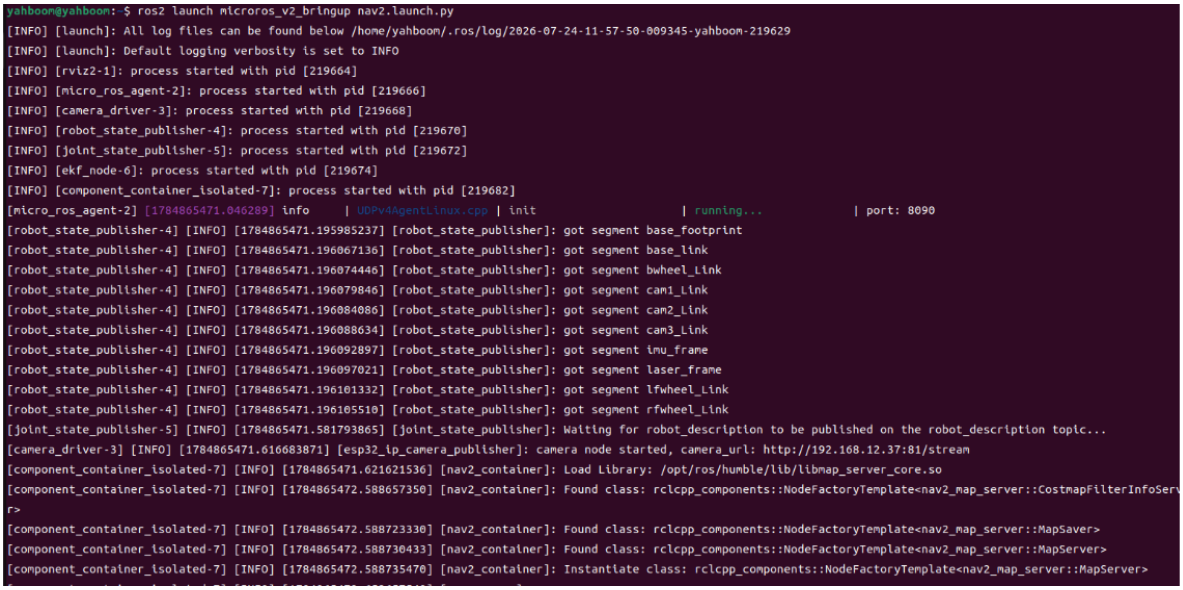
```
sh ~/bringup_dify.sh
```

3. Running the Case

3.1 Starting the Program

Open the virtual machine terminal and enter the command to start the underlying proxy + camera topic + navigation:

```
ros2 launch microros_v2_bringup nav2.launch.py
```



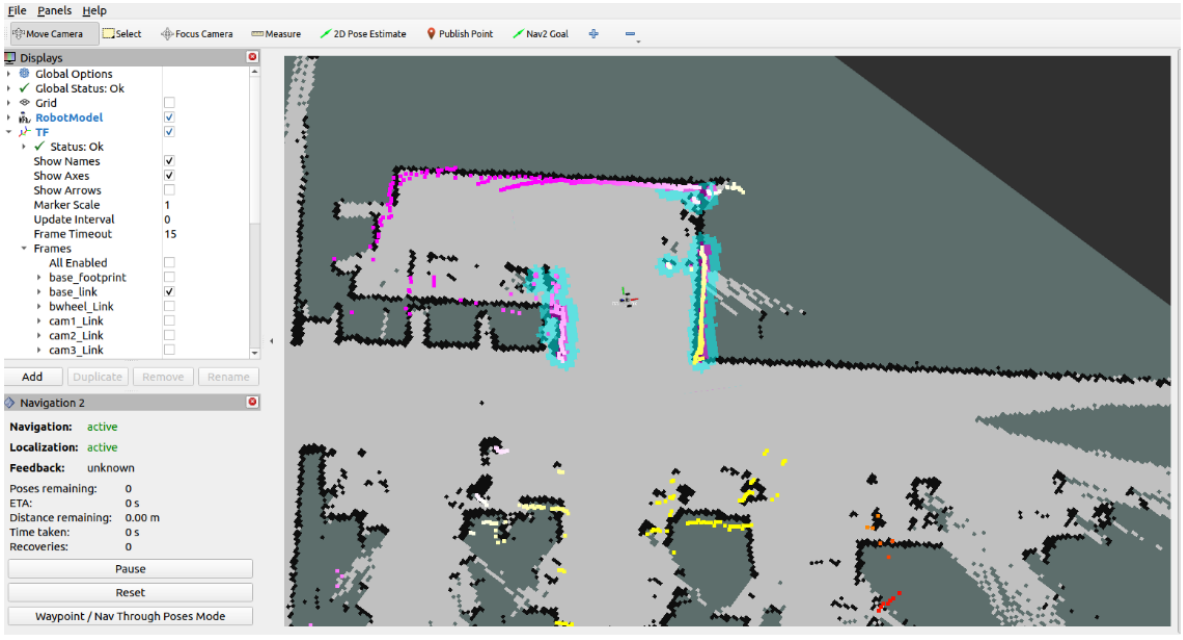
Open another virtual machine terminal and enter the command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization is complete, the following content will be displayed:

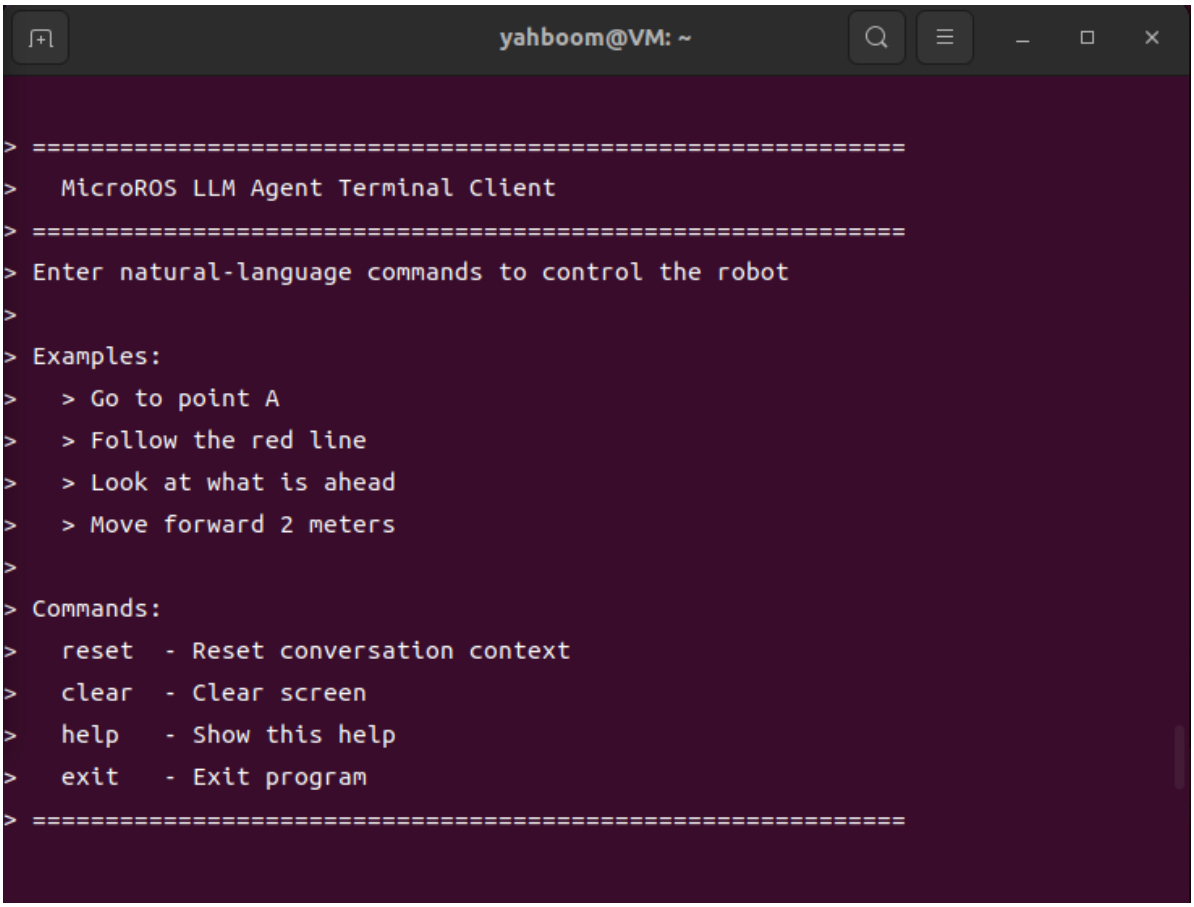
```
yahboom@VM: ~
G_MODE=False, LANGUAGE=en, CAR_TYPE=ptz
[llm_agent_node-1] [INFO] [1787128762.059479166] [llm_agent_node]: Initializing
LLM Agent Node...
[llm_agent_node-1] [INFO] [1787128762.070369334] [llm_agent_node]: [CONFIG] cmd_
vel_topic=/cmd_vel
[llm_agent_node-1] [INFO] [1787128762.077274270] [llm_agent_node]: [CONFIG] imag
e_cache_path=/home/yahboom/microros_ws/cache/llm_camera.jpg
[llm_agent_node-1] [INFO] [1787128762.078819208] [llm_agent_node]: [PTZ] startup
 initialization enabled: x=0.0°, y=-45.0°, count=20, interval=0.10s
[llm_agent_node-1] [INFO] [1787128762.106861388] [llm_agent_node]: [FOLLOW] scan
_topic=/scan, obstacle_guard=True, buzzer_topic=/beep, buzzer=True
[llm_agent_node-1] [INFO] [1787128762.107460365] [llm_agent_node]: [DIFY] Initia
lize client: base_url=http://localhost/v1, api_key=app-SAnX****Zt2E
[llm_agent_node-1] [INFO] [1787128762.109741022] [llm_agent_node]: [DIFY] Testin
g connection....
[llm_agent_node-1] [INFO] [1787128762.130633809] [llm_agent_node]: [TRACK_KCF] S
ubscribed /tracking_bbox, type=smart_tracker/msg/TrackBox
[llm_agent_node-1] [INFO] [1787128762.131212772] [llm_agent_node]: LLM Agent Nod
e initialized successfully
[llm_agent_node-1] [INFO] [1787128762.133300019] [llm_agent_node]: [DIFY] Connec
tion successful ✓
[llm_agent_node-1] [INFO] [1787128764.080740503] [llm_agent_node]: [PTZ] Servo i
nitialization complete: x=0.0°, y=-45.0°
```

Then follow the process of starting the navigation function for initial localization. After opening the RViz2 visualization interface, click **2D Pose Estimate** in the toolbar at the top, roughly mark the robot's position and orientation on the map. After initial localization, the preparation is complete.



Then open another terminal and start the text dialogue:

```
ros2 run multi_brains terminal_client
```



3.2 Test Cases

Reference test cases are given here. Users can create their own dialogue commands.

- Record the current position, navigate to the tea room and observe whether there is a red line on the ground. If there is a red line, automatically follow the red line. After line following ends, return to the starting point.

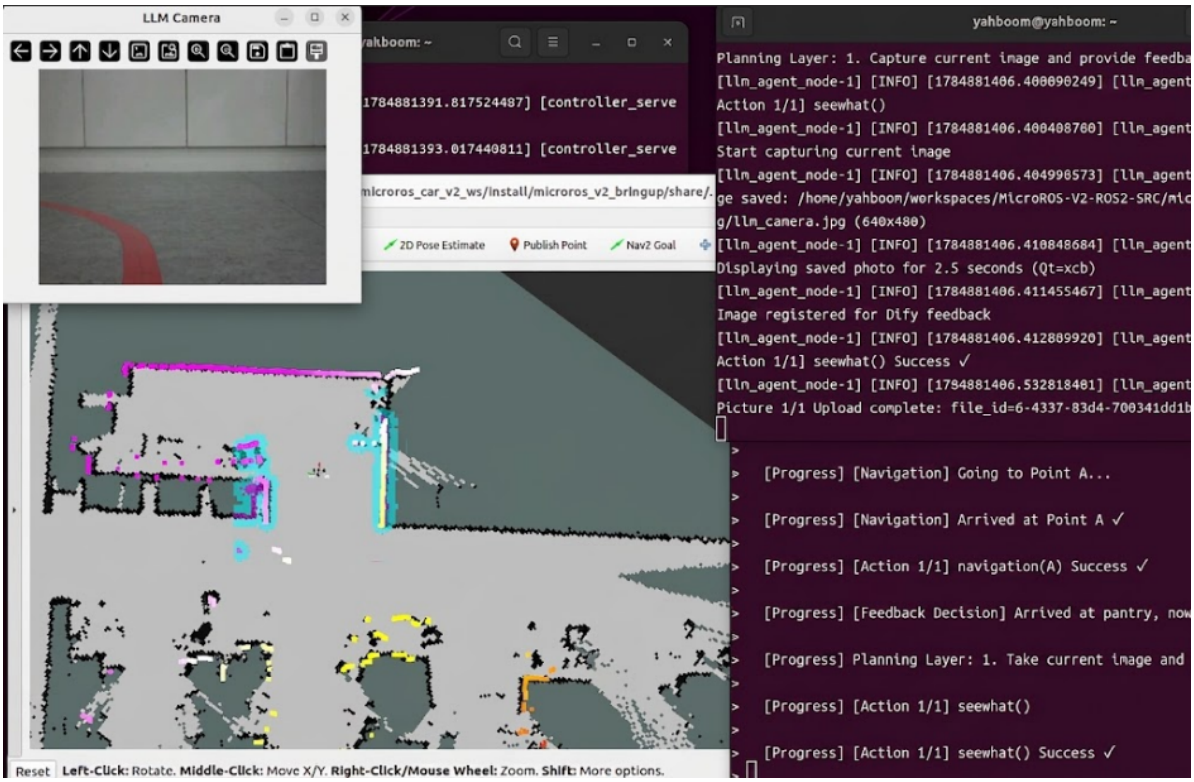
3.2.1 Case 1

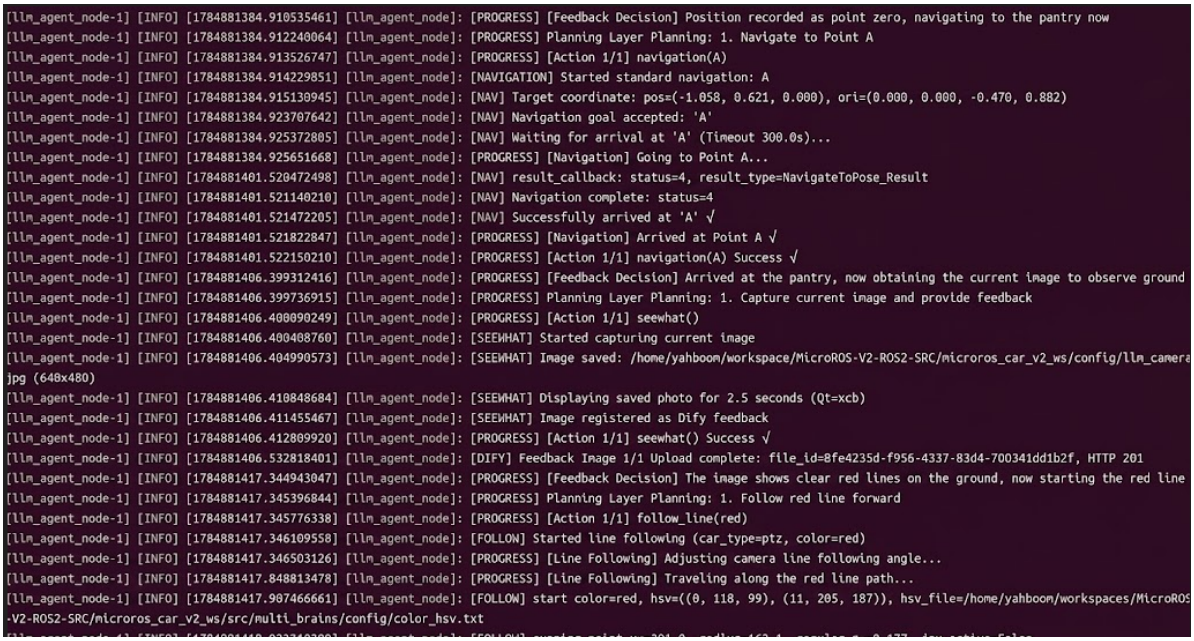
Enter "Record the current position, navigate to the tea room and observe whether there is a red line on the ground. If there is a red line, automatically follow the red line. After line following ends, return to the starting point." in the terminal. The terminal prints information as follows:

```
> [Sending] Record current position, navigate to pantry to observe if there's a red line on the
ground. If so, perform automatic red-line patrolling. After patrolling, return to starting point.
> [Sent] New commands can be entered; the latest will take precedence.

>
>   [Progress] [Decision] Thinking...
>
>   [Progress] [Decision] Start executing task. First, record current position as starting point.
>
>   [Progress] Decision-level planning: 1. Obtain current position coordinates.
>
>   [Progress] [Preparation] Adjusting camera to face directly ahead...
>
>   [Progress] [Action 1/1] get_current_pose()
>
>   [Progress] [Action 1/1]  get_current_pose() Successful √
>
```

Then the execution-layer large model will execute according to the decision-layer steps:





After the robot completes the task, it enters a waiting state. At this time, instructions will continue to be passed to the execution-layer large model, and the conversation history will be retained. You can continue the dialogue in the terminal and input the "/reset" command to let the robot end the current task cycle and start a new task cycle.

```
[llm_agent_node-1] [INFO] [1784867518.113324588] [llm_agent_node]: [INPUT] ── Received user input #3: 'res
set'
[llm_agent_node-1] [INFO] [1784867518.114754858] [llm_agent_node]: [PROGRESS] [Decision] Thinking...
[llm_agent_node-1] [INFO] [1784867522.880420933] [llm_agent_node]: [ACTION] finishtask: Task cycle finished
[llm_agent_node-1] [INFO] [1784867522.881048600] [llm_agent_node]: [RESULT] System reset, awaiting new comm
ands
[llm_agent_node-1] [INFO] [1784867522.881456002] [llm_agent_node]: [INPUT] ── Instruction #3 processing co
12, total time: 4.77s ──
```

4. Source Code Analysis

This compound case chains together chassis motion, current position recording, SLAM navigation, visual understanding, and visual line following. In the current version, `llm_agent_node.py` schedules uniformly, executing only one action per round, and after the action is completed, the result is fed back to Dify, which decides the next step.

Current relevant source code paths:

```
~/microros_ws/src/multi_brains/multi_brains/llm_agent_node.py
~/microros_ws/src/multi_brains/multi_brains/controllers/motion_controller.py
~/microros_ws/src/multi_brains/multi_brains/controllers/navigation_controller.py
~/microros_ws/src/multi_brains/multi_brains/controllers/follow_line_controller.p
y
```

Possible action sequence:

```
get_current_pose()
navigation(target)
seewhat()
follow_line(green)
navigation(zero)
finishtask()
```

Key points:

- `get_current_pose()` records the current position as the runtime dynamic point `zero`.
- `navigation(target)` uses the target point coordinates in `config/map_mapping.yaml`.
- `seewhat()` uploads the current frame to Dify to determine whether there is a specified color line on the ground.
- `follow_line(color)` reads the color calibration values in `config/color_hsv.txt` and controls the chassis line following.
- If a new instruction is input at any stage, the current action is preempted by the latest instruction, and navigation, line following, and chassis actions are stopped as soon as possible.